Claude Code Onboarding Plan

Nitin Vavdiya · Solution Architect · 160 Hours

Role: Solution Architect

Company: smartSense Consulting Solutions Pvt. Ltd.

Current Projects: Cofinity-X · Keenlyco · CoSSBot · Certif-ID · smartProof · The Doo · KEVA Health

Tech Stack: Java · Spring Boot · Spring Security · Spring Batch · Spring Cloud Gateway · PostgreSQL · MariaDB · Redis · ElasticSearch · Keycloak · Docker · Kubernetes · AWS · Azure · Argo Workflow · OAuth2 · Neo4j · Helm · Python

Domains: Automotive Data Spaces · SSI/DID · Blockchain Credentialing · ESG Reporting · Health Tech · Smart Home Automation

Plan Type: Single integrated plan — basic to pro level

Total Hours: 160 hrs · 10 milestones · 40 tasks

A Note on This Plan

Nitin is a Solution Architect with 12 years of experience across some of the most technically complex domains in modern software — decentralised identity, automotive data spaces, blockchain credentialing, and cloud-native microservices. He already uses Cursor, which means he has baseline AI tool familiarity and will move through the orientation phase faster than most.

For Nitin, Claude Code is not a code completion tool — it is an architectural thinking partner, a multi-service code generation engine, a documentation accelerator, and a team leadership multiplier. He designs systems that other engineers build. Claude Code helps him move from architecture to implementation faster, document decisions more thoroughly, and scale his technical leadership across teams.

Every task in this plan is anchored to his real current projects — Cofinity-X, Certif-ID, SSI/DID workflows, and the broader smartSense technology stack.

Milestone 1: Orientation & Setup

10 hrs · 4 tasks *Experience goal: "I understand how Claude Code differs from Cursor and other AI tools I already use, and I can apply it to my architecture and development work immediately."*

Task 1 · 1.5 hrs

Watch & Read: Claude Code vs. Cursor — Understanding the Difference

You already use Cursor. The most important thing to establish upfront is what Claude Code does that Cursor does not — and why both have a place in your workflow. This is not a beginner orientation; this is a tool differentiation exercise for an experienced practitioner.

Resources

- Claude Code Overview (Official Docs): https://docs.anthropic.com/en/docs/claude-code/overview
- Claude Code in Action — Module 1 (Anthropic Academy): https://anthropic.skilljar.com/claude-code-in-action/303233
- Claude Code Without the Code (Nate's Newsletter): https://natesnewsletter.substack.com/p/claude-code-without-the-code-the

Exercise Read and watch both resources. Then write in your Notes: (1) Three specific workflows where Claude Code would outperform Cursor for your architecture and multi-service development work. (2) Three workflows where you would still prefer Cursor. (3) How you would describe the difference to a junior developer on your team who only knows Cursor.

Notes for Nitin Given your Cursor experience, resist the temptation to use Claude Code the same way you use Cursor — as a contextual code assistant. Claude Code's strength is in reasoning across your entire codebase, executing multi-step tasks, and handling the full workflow from architecture decision to committed code. Think of it as the difference between a highly capable autocomplete and a highly capable engineer sitting beside you who can read your whole repository, not just the current file.

Task 2 · 1.5 hrs

Setup: Install Claude Code and Run Your First Session on a Real Project

Get Claude Code installed and immediately test it on one of your active projects — Cofinity-X or Certif-ID. Your first session should not be a tutorial; it should be a real architectural conversation about a system you designed.

Resources

- Official Quickstart Guide: https://docs.anthropic.com/en/docs/claude-code/quickstart
- Installation Troubleshooting FAQ: https://docs.anthropic.com/en/docs/claude-code/faq

Exercise Install Claude Code. Run `claude --version` and `/doctor` to confirm. Navigate to your primary project repository. Run `claude`. Ask: *"Analyse this project's architecture — describe the main services, how they communicate, the key design decisions you can infer from the code structure, and any architectural concerns you would raise if you were doing a review."* Evaluate the depth and accuracy of Claude's architectural analysis.

Notes for Nitin You are testing Claude's architectural comprehension from the first session. With Cofinity-X's complexity — AKS, ArgoCD, Keycloak IDPs, Managed Identity Wallet — Claude will get some things right and some things wrong. The gaps between what it understands and what it misses are exactly the things your CLAUDE.md needs to capture.

Task 3 · 3 hrs

Study: Core Concepts — Context Window, Plan Mode, and the Approval Flow

Even with Cursor experience, Claude Code's core concepts work differently enough to warrant dedicated study. The context window behaviour, the approval flow, and Plan Mode are the three things that most experienced AI tool users underestimate.

Resources

- Claude Code in Action — Modules 1 & 2 (Anthropic Academy): https://anthropic.skilljar.com/claude-code-in-action/303233
- Core Concepts — GitHub Ultimate Guide Section 2: https://github.com/FlorianBruniaux/claude-code-ultimate-guide/blob/main/guide/ultimate-guide.md#2-core-concepts
- Session Management (Official Docs): https://docs.anthropic.com/en/docs/claude-code/session-management

Exercise Complete both Anthropic Academy modules. Then write in your Notes: (1) How does the context window constraint change your strategy for working across a large multi-service repository like Cofinity-X? (2) How does the approval flow compare to your existing PR review process — and how would you use it as a quality gate? (3) When would you use /plan before a task, and give three specific examples from your current project work? (4) When would you use /compact versus /clear in a session focused on architecture design?

Notes for Nitin The context window is the most critical constraint for your work. A large multi-service project like Cofinity-X with Kubernetes manifests, multiple Spring Boot services,

Terraform configs, and Helm charts can easily exceed context limits. The correct approach: one service per session, always. Your CLAUDE.md handles the cross-service context that Claude cannot hold in memory.

Task 4 · 4 hrs

Study: Advanced Prompting for Architects — Directing Complex Multi-Service Work

As a Solution Architect, the prompting skills that matter most for you are different from those of a typical developer. Your prompts need to convey architectural intent, cross-service constraints, security requirements, and domain-specific complexity — particularly for your SSI/DID and automotive data space work.

Resources

- Prompt Engineering Overview (Anthropic): https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview
- Claude Code Best Practices (Official Docs): https://docs.anthropic.com/en/docs/claude-code/best-practices
- Nick Saraev's Claude Code Masterclass — first 90 minutes (YouTube): https://www.youtube.com/watch?v=6VakQSJFTZo

Exercise Read and watch all resources. Build a personal list of at least 10 Prompting Principles in your Notes, specifically for Solution Architecture work. Examples to get started:

- "Always specify the target deployment environment (AKS, AWS EKS, DigitalOcean) before asking for Kubernetes manifests or Helm charts"
- "Always describe the security model (OAuth2 flow, Keycloak realm structure, DID method) before asking for authentication-related code"
- "Always specify the inter-service communication pattern (REST, event-driven, MQTT) before asking for integration code" Add at least 7 more specific to your domains — SSI/DID, Spring Security, Argo Workflow, Catena-X components.

Notes for Nitin For your SSI and blockchain work specifically, Claude Code needs heavy domain context upfront. DID methods, verifiable credentials, and Catena-X protocols are not widely trained patterns — you will need to include more context in your prompts and CLAUDE.md than you would for standard Spring Boot work. The payoff is significant: once the context is established, Claude can generate boilerplate, documentation, and integration code for these complex domains faster than any other tool.

Milestone 2: CLAUDE.md — Architectural Context Files

10 hrs · 4 tasks *Experience goal: "Claude understands the architectural decisions, technology choices, and cross-cutting concerns of my projects without me re-explaining them every session."*

Task 5 · 2 hrs

Study: CLAUDE.md for Architects — Beyond Basic Project Context

For a Solution Architect, CLAUDE.md is not just a list of tech stack items. It is an architectural brief — the document that encodes design decisions, integration contracts, security models, and the reasoning behind architectural choices so Claude can generate code and documentation that respects them.

Resources

- CLAUDE.md Documentation (Official Docs): https://docs.anthropic.com/en/docs/claude-code/memory
- CLAUDE.md Deep Dive — GitHub Ultimate Guide Section 3.1: https://github.com/FlorianBruniaux/claude-code-ultimate-guide/blob/main/guide/ultimate-guide.md#31-memory-files-claudemd

Exercise Read both resources. Write in your Notes the 10–12 things about one of your current projects (Cofinity-X or Certif-ID) that you would most want Claude to understand before every session. Think beyond tech stack: include architectural patterns, security model, inter-service contracts, deployment conventions, and the domain-specific vocabulary that Claude is unlikely to infer from the code alone (e.g. what EDC means, what Managed Identity Wallet does, what the Catena-X data space architecture looks like).

Notes for Nitin Your CLAUDE.md for a Catena-X project needs a section that no standard template will include: a brief on the IDS/GAIA-X data space concepts, the EDC connector role, and the Managed Identity Wallet's relationship to DID. Without this, Claude will treat these as generic Spring Boot services and generate contextually wrong code. With it, Claude generates code that understands the protocol context it is operating in.

Task 6 · 3 hrs

Exercise: Build CLAUDE.md for Your Primary Architecture Project

Create a production-quality CLAUDE.md for your most architecturally complex current project — Cofinity-X or Certif-ID. This should be the most detailed CLAUDE.md you have ever written for any tool.

Resources No external resources required — this is a deep-knowledge creation exercise.

Exercise Create a CLAUDE.md in your primary project folder. Include: (1) Project overview — domain context, what problem it solves, who uses it, and the key architectural decisions already made. (2) Architecture overview — microservice boundaries, how services communicate, shared libraries, API gateway configuration. (3) Tech stack with versions — Spring Boot version, Java version, Kubernetes version, Keycloak version, database per service. (4) Security model — OAuth2 flows, Keycloak realm structure, DID/SSI integration points if applicable. (5) Deployment conventions — Kubernetes namespace structure, Helm chart conventions, Argo Workflow patterns, environment naming. (6) Coding conventions — package structure, exception handling patterns, logging standards, test framework conventions. (7) What Claude should never do — e.g. never modify shared library interfaces, never add new service-to-service synchronous calls without noting it, never generate code that bypasses the API gateway. (8) Domain vocabulary — a brief glossary of project-specific terms (EDC, Managed Identity Wallet, DID, Verifiable Credentials, etc.).

Notes for Nitin The domain vocabulary section is your highest-value investment. Claude Code has baseline knowledge of Spring Boot, Kubernetes, and OAuth2. It does not have deep knowledge of Catena-X protocols, SSI architectures, or your specific Keycloak realm design. A 10-line glossary in your CLAUDE.md changes the quality of every session that touches these domains.

Task 7 · 2 hrs

Exercise: Test and Refine Your CLAUDE.md

Verify that a fresh session with only your CLAUDE.md loaded produces architectural understanding, not generic responses.

Resources No external resources required.

Exercise Run `/clear`. Without additional context, ask Claude: (1) "What is the security model for this project — how does authentication work across services?" (2) "If I need to add a new microservice to this system, what conventions must it follow?" (3) "What is the role of [project-specific component, e.g. the EDC connector or Managed Identity Wallet] in this architecture?" (4) "What should you never change in this codebase without my explicit approval?" Grade each answer against your CLAUDE.md. Refine until every answer reflects your actual architecture accurately.

Notes for Nitin If Claude answers the security model question generically (e.g. "Spring Security with JWT tokens") rather than specifically (e.g. "OAuth2 via Keycloak, realm X, with service-to-service auth via client credentials flow"), your CLAUDE.md's security section needs

more specificity. Rewrite it with concrete examples — the actual realm name, the actual flow type, the actual token validation approach.

Task 8 · 3 hrs

Exercise: Build CLAUDE.md Files for Secondary Projects and Global Preferences

Create CLAUDE.md files for your secondary projects and a global personal CLAUDE.md that encodes your architectural preferences across all work.

Resources

- CLAUDE.md Hierarchy (Official Docs): https://docs.anthropic.com/en/docs/claude-code/memory

Exercise (1) Create a CLAUDE.md for a second active project — Certif-ID or Keenlyco — following the same structure as Task 6. (2) Create a global CLAUDE.md at `~/.claude/CLAUDE.md` encoding your personal architectural standards: your preferred exception handling approach, your logging standards, your API design conventions, your Kubernetes resource naming convention, your preferred OAuth2 implementation pattern, and your stance on synchronous vs. asynchronous inter-service communication. (3) Test the global CLAUDE.md by starting a session on a project that does not have a project-level CLAUDE.md. Verify your personal preferences are applied.

Notes for Nitin Your global CLAUDE.md captures the architectural principles you have developed across 12 years of work — things like "always use async messaging for cross-domain events," "never expose internal service ports outside the cluster," "always version REST APIs from v1." These standards should not need to be restated per project. Put them in your global file once and they apply everywhere.

Milestone 3: First Win — Real Architectural Work

14 hrs · 4 tasks *Experience goal: "I used Claude Code to complete a real architecture or development task that I would normally have done manually, and the output saved me significant time."*

Task 9 · 2 hrs

Study: The Complete Anthropic Academy Course

Complete the full official Claude Code course with a focus on the capabilities most relevant to your Solution Architect role — multi-file editing, Plan Mode for complex tasks, and Git workflow automation.

Resources

- Claude Code in Action — Full Course (Anthropic Academy): https://anthropic.skilljar.com/claude-code-in-action/303233
- Claude Code: A Highly Agentic Coding Assistant (DeepLearning.AI): https://www.deeplearning.ai/short-courses/claude-code-a-highly-agentic-coding-assistant/

Exercise Complete both courses. After each module, write one sentence: "The most applicable capability in this module for my Solution Architect work is ___." Pay particular attention to multi-file editing and Git workflow automation — these are the sections with the highest direct value for your role.

Notes for Nitin At your experience level, the highest-value sections are the ones on agentic multi-step execution and Git integration. Claude Code's ability to plan a change across multiple services, execute it step by step, and generate the commit and PR description is where you will see the most dramatic time savings compared to your current workflow.

Task 10 · 2 hrs

Identify Your First Real Task

Choose one real task from your current project work that you would normally do manually. For a Solution Architect, this should be something that currently takes significant time but follows a pattern — boilerplate service creation, documentation generation, or configuration templating.

Resources No external resources required — review your current Jira backlog.

Exercise Browse your Jira board. Identify one task that: (a) is in your current sprint, (b) is scoped to 1–3 hours of work, (c) produces something concrete and reviewable. Strong candidates for your role:

- Scaffolding a new Spring Boot microservice following your project's conventions
- Generating or updating API documentation for an existing service
- Creating Kubernetes manifests or a Helm chart for a new component
- Documenting a new architectural decision (ADR)
- Writing integration code for a Keycloak custom flow or SSI component Write the task and what "done and correct" looks like.

Notes for Nitin Given your experience, choose something where you already know the correct output in detail. Your First Win is most compelling when you can say "Claude got this right on the first or second prompt and it would have taken me two hours manually." Scaffolding a new microservice that follows all your project conventions — package structure, exception handling, logging, Swagger, Docker, Kubernetes manifest — is an ideal First Win candidate.

Task 11 · 4 hrs

Exercise: Complete Your First Real Architecture Task

Use Claude Code with your CLAUDE.md loaded to complete the real task you identified.

Resources No external resources required — this is real project work.

Exercise Open a session in your project repository. Confirm Claude has read your CLAUDE.md. Then complete your chosen task. As you work: (1) Note every place where Claude's output respected your architectural conventions without being explicitly told — what did your CLAUDE.md successfully communicate? (2) Note every place where the output needed correction — what was missing from your CLAUDE.md? (3) Count how many prompts it took to get production-ready output. When done, write your First Win story: what you built, how long it took versus manually, and what you would add to your CLAUDE.md before the next similar task.

Notes for Nitin For service scaffolding especially: use /plan first and ask Claude to describe the full file structure it will create before generating any code. As a Solution Architect, reviewing the plan takes you 2 minutes and prevents 30 minutes of correcting a structure that was built in the wrong direction.

Task 12 · 6 hrs

Study: Advanced Workflows and Common Patterns

Study documented Claude Code workflows and adapt them to the specific patterns of Solution Architecture work — service design, integration implementation, and technical documentation.

Resources

- Common Workflows (Official Docs): https://docs.anthropic.com/en/docs/claude-code/common-workflows
- Claude Code Beginner's Tutorial — Build a Movie App (Peter Yang, YouTube): https://www.youtube.com/watch?v=zxMjOqM7DFs
- Nick Saraev's Claude Code Masterclass — first 90 minutes (YouTube): https://www.youtube.com/watch?v=6VakQSJFTZo

- Claude Code Everything You Need to Know — Sections 1–3 (GitHub): https://github.com/wesammustafa/Claude-Code-Everything-You-Need-toKnow/blob/main/README.md

Exercise Watch and read all resources. Identify 5 workflows directly applicable to your Solution Architect role. For each, write a one-paragraph adaptation: "Here is how I would apply this on Cofinity-X or Certif-ID." Pay particular attention to the multi-service editing workflow and the architectural documentation workflow.

Notes for Nitin The pattern that will change your daily work most: Plan → Execute one service at a time → Review output → Commit → Move to next service. For cross-cutting changes (adding a new header to all service API responses, updating a shared library version across all services), Claude Code can execute this across multiple files in a single planned task — something that currently takes significant time to do manually.

Milestone 4: Microservice Development Acceleration

18 hrs · 4 tasks *Experience goal: "I can use Claude Code to generate, extend, and refactor Spring Boot microservices at a speed that meaningfully accelerates delivery across my team."*

Task 13 · 4 hrs

Exercise: Complete Microservice Scaffolding Workflow

Build a reliable Claude Code workflow that generates a complete, production-quality Spring Boot microservice scaffold following your project conventions in a single directed session.

Resources

- Spring Boot Documentation: https://docs.spring.io/spring-boot/docs/current/reference/html/
- Spring Security Documentation: https://docs.spring.io/spring-security/reference/

Exercise Design a prompt sequence that generates a complete new Spring Boot microservice for one of your projects, including: project structure (packages, modules), pom.xml with correct dependencies, application.yml and application-[env].yml, Dockerfile (multi-stage), Kubernetes deployment manifest, Spring Security configuration (OAuth2 / Keycloak integration), exception handling setup, logging configuration, Swagger/OpenAPI setup, and a base health check endpoint. Test on a new service you need for a current project. Measure quality on first attempt.

Notes for Nitin The key to reliable microservice scaffolding is your CLAUDE.md — specifically the sections on package structure, exception handling patterns, and Keycloak integration. Once these are detailed enough, Claude generates a scaffold that follows your conventions from the first prompt. If the Keycloak integration section in your CLAUDE.md includes your actual realm name and client ID pattern, Claude will generate the Spring Security config correctly without needing to be told.

Task 14 · 4 hrs

Exercise: Spring Security and Keycloak Integration Workflows

Build a repeatable Claude Code workflow for implementing and auditing Spring Security and Keycloak integrations — one of the most complex and error-prone areas of your current work.

Resources

- Keycloak Documentation: https://www.keycloak.org/docs/latest/
- Spring Security OAuth2 Documentation: https://docs.spring.io/spring-security/reference/servlet/oauth2/

Exercise Choose a Keycloak integration task from your current project — a new resource server configuration, a custom token claim mapper, or a new realm-level role policy. Use /plan first. Ask Claude to: (1) explain the OAuth2 flow for this integration in plain language before writing a line of code, (2) identify all Spring Security filter chain components it will create or modify, (3) identify the Keycloak realm-side configuration that must exist for this to work. Only after approving the plan, generate the implementation. Review every security-relevant line carefully.

Notes for Nitin Spring Security configuration is the highest-risk area for subtle but serious errors in your work. Claude can generate plausible-looking Spring Security code that is logically incorrect — wrong filter order, wrong token validation logic, missing CSRF configuration for stateful endpoints. Always review every line of generated Spring Security code as if it were a security audit, not just a code review.

Task 15 · 5 hrs

Exercise: SSI/DID and Blockchain Credentialing Workflows

Use Claude Code as a development accelerator for your most domain-specific work — Self-Sovereign Identity, Decentralised Identifiers, Verifiable Credentials, and Catena-X integrations.

Resources

- W3C DID Specification: https://www.w3.org/TR/did-core/
- W3C Verifiable Credentials: https://www.w3.org/TR/vc-data-model/
- Eclipse Tractus-X Documentation: https://eclipse-tractusx.github.io/

Exercise Choose a task from your SSI or Catena-X work — a new DID document handler, a Verifiable Credential issuance flow, or an EDC connector integration point. Before generating any code, ask Claude to: (1) summarise its understanding of the relevant W3C or Catena-X specification for this task, (2) identify where your implementation will differ from the standard due to your specific setup, (3) list all dependencies and external services this component will touch. Evaluate: how accurate is Claude's understanding of SSI/DID concepts? Where does it need correction? This calibration is essential before using it for domain-specific code generation.

Notes for Nitin Claude Code has general knowledge of DID and SSI concepts but limited specific knowledge of Eclipse Tractus-X project conventions and Catena-X protocol specifics. Your CLAUDE.md's domain vocabulary section does the heavy lifting here. The more context you provide about your specific implementation approach (which DID method, which credential format, which EDC version), the more accurate the generated code will be.

Task 16 · 5 hrs

Exercise: Multi-Service Refactoring and Cross-Cutting Changes

Use Claude Code to execute cross-cutting changes across multiple microservices — one of the most time-consuming tasks in a microservice architect's workflow.

Resources

- Claude Code Common Workflows (Official Docs): https://docs.anthropic.com/en/docs/claude-code/common-workflows

Exercise Identify a cross-cutting change that needs to be applied across multiple services in one of your projects — for example, updating a shared library version, adding a new correlation ID header to all service APIs, standardising exception handling across services, or updating Kubernetes resource limits across all deployments. Use /plan first and ask Claude to map all affected services and files before making any changes. Execute service by service, reviewing each change before moving to the next. Document: how many files were changed correctly on first attempt?

Notes for Nitin Cross-cutting changes are where Claude Code delivers the most dramatic time savings for Solution Architects. A change that previously required you to touch 12 service repositories manually — each one a separate PR, each one requiring context switching —

becomes a directed, systematic session where Claude executes the pattern and you review. The discipline: never approve a cross-cutting change without reading the plan first. The plan surfaces edge cases you would otherwise discover mid-execution.

Milestone 5: Architecture Documentation and Decision Records

14 hrs · 4 tasks *Experience goal: "I use Claude Code to create, maintain, and improve the architectural documentation that enables my team to build with confidence."*

Task 17 · 3 hrs

Exercise: Architecture Decision Records (ADR) Generation

Use Claude Code to rapidly generate professional Architecture Decision Records for the most significant technical decisions in your projects.

Resources

- Architecture Decision Records Format: https://adr.github.io/
- Michael Nygard's Original ADR Format: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Exercise Identify 3–5 significant architectural decisions made for your current projects — for example: "Why Spring Cloud Gateway over Nginx for the API gateway?", "Why Keycloak over Auth0 for identity?", "Why Argo Workflow for orchestration?", "Why PostgreSQL per service rather than a shared database?". For each, ask Claude to generate an ADR documenting the decision, alternatives considered, consequences, and rationale. Review and correct with your actual reasoning. Save these as permanent documentation.

Notes for Nitin ADRs are one of the most neglected but most valuable documents in a microservice project. When a new architect joins your team or a client asks "why was this designed this way?", an ADR gives the complete answer immediately. Claude Code makes generating them fast — you provide the reasoning, Claude structures it professionally. Your 12 years of architectural decisions across Cofinity-X, Certif-ID, and CoSSBot represent a significant body of knowledge that deserves to be documented this way.

Task 18 · 3 hrs

Exercise: System Architecture Documentation Generation

Use Claude Code to generate and maintain comprehensive system architecture documentation from your actual codebase and infrastructure configurations.

Resources No external resources required — apply to your current project.

Exercise For one of your active projects, ask Claude to read your codebase and generate: (1) a service inventory — all services, their purpose, and their primary responsibilities, (2) a service dependency map — which services call which, and how, (3) a data flow description for your two most critical business workflows, (4) an external dependency inventory — all third-party integrations, APIs, and services the system depends on. Review and correct the output. Add missing context to your CLAUDE.md so subsequent sessions produce more accurate documentation.

Notes for Nitin This documentation exercise produces something your whole team needs but rarely has time to create — an accurate, up-to-date system overview. For Cofinity-X especially, where the architecture involves Catena-X components, Keycloak IDPs, and Managed Identity Wallet integrations that are not self-explanatory from the code, Claude-generated documentation with your expert review and correction is significantly faster than writing it from scratch.

Task 19 · 4 hrs

Exercise: API Contract Documentation and Review

Use Claude Code to generate, audit, and maintain OpenAPI specifications and API contract documentation across your microservices.

Resources

- OpenAPI 3.0 Specification: https://swagger.io/specification/
- SpringDoc OpenAPI: https://springdoc.org/

Exercise For 2–3 services in your current project: (1) Ask Claude to audit the existing Swagger/OpenAPI annotations and identify missing or incomplete documentation. (2) Ask Claude to generate complete OpenAPI annotations for undocumented endpoints. (3) Ask Claude to generate a human-readable API contract document for each service — suitable for sharing with client integration teams. (4) Ask Claude to identify any REST design inconsistencies across services (different error response formats, inconsistent naming conventions, different status code usage). Document and fix the inconsistencies found.

Notes for Nitin API contract consistency across microservices is one of the hardest things to maintain in a growing system. Using Claude Code to audit all services against a consistent API

design standard — and generate the corrections — is a high-leverage activity that typically reveals inconsistencies that accumulated silently over multiple sprints.

Task 20 · 4 hrs

Exercise: Technical Specification and RFC Generation

Use Claude Code to generate technical specifications, RFC documents, and implementation briefs for upcoming features — reducing the time from architectural decision to team-ready specification.

Resources No external resources required — use upcoming features from your roadmap.

Exercise Take 2–3 upcoming features or architectural changes from your roadmap that require a technical specification before development begins. For each: (1) Describe the business requirement and architectural context to Claude. (2) Ask Claude to generate a complete technical specification including: affected services, required code changes, new components needed, security considerations, deployment changes, and estimated complexity. (3) Ask Claude to identify the questions the development team will ask that the specification does not yet answer. (4) Review, correct, and finalise the specification. Measure time saved versus writing from scratch.

Notes for Nitin This is the workflow that has the highest impact on your team's velocity. A well-written technical specification that Claude generates in 30 minutes — and you review and refine in 30 minutes — saves your development team hours of clarification cycles and rework. For complex SSI or Catena-X features especially, the specification that comes out of this workflow becomes the document that makes implementation predictable rather than exploratory.

Milestone 6: DevOps, Cloud, and Infrastructure Workflows

16 hrs · 4 tasks *Experience goal: "I use Claude Code to generate, review, and maintain the Kubernetes, Helm, and cloud infrastructure configurations that my team deploys."*

Task 21 · 4 hrs

Exercise: Kubernetes Manifest and Helm Chart Generation

Use Claude Code to generate and maintain production-quality Kubernetes manifests and Helm charts for your microservices across AKS, AWS EKS, and DigitalOcean.

Resources

- Kubernetes Documentation: https://kubernetes.io/docs/home/
- Helm Documentation: https://helm.sh/docs/

Exercise For a service that needs new or updated Kubernetes manifests: (1) Ask Claude (with your CLAUDE.md loaded) to generate a complete set of Kubernetes manifests — Deployment, Service, ConfigMap, Secret template, HPA, and NetworkPolicy — following your project's namespace and labelling conventions. (2) Ask Claude to generate the equivalent Helm chart with values.yaml structured for multi-environment deployment (dev, staging, production). (3) Ask Claude to review the generated manifests for security issues — overly permissive RBAC, missing resource limits, insecure environment variable handling. Document quality and corrections needed.

Notes for Nitin Your CLAUDE.md's deployment conventions section is critical for this task. Include: your namespace naming pattern, your label structure (app, version, component, managed-by), your resource limit standards per service tier, your Argo CD application naming convention, and whether services use ClusterIP or NodePort. Without these, Claude generates technically valid but convention-violating manifests.

Task 22 · 4 hrs

Exercise: Argo Workflow and CI/CD Pipeline Generation

Use Claude Code to generate and maintain your Argo Workflow definitions and GitHub Actions / GitLab CI pipeline configurations.

Resources

- Argo Workflows Documentation: https://argoproj.github.io/argo-workflows/
- GitHub Actions Documentation: https://docs.github.com/en/actions

Exercise Choose an Argo Workflow or CI/CD pipeline that needs to be created or updated. Use /plan first. Ask Claude to: (1) Design the workflow steps in plain language before generating any YAML. (2) Generate the full workflow definition following your existing patterns. (3) Add appropriate error handling, retry logic, and notification steps. (4) Generate the corresponding GitHub Actions or GitLab CI trigger for the workflow. Review every step of the generated workflow against your deployment standards.

Notes for Nitin Argo Workflow YAML is one of the most verbose and error-prone things to write manually. Claude generates structurally correct Argo Workflow definitions reliably when you

specify: the workflow type (DAG vs. steps), the container images to use, the artifact storage configuration, and the notification/alerting integration. Include your Argo Workflow template patterns in your CLAUDE.md.

Task 23 · 4 hrs

Exercise: Cloud Infrastructure Review and Documentation

Use Claude Code to audit, document, and improve your cloud infrastructure configurations across AWS and Azure.

Resources

- AWS Well-Architected Framework: https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html
- Azure Well-Architected Framework: https://learn.microsoft.com/en-us/azure/architecture/framework/

Exercise For your current cloud infrastructure (AWS or Azure): (1) Ask Claude to read your Terraform or CloudFormation configurations and generate an infrastructure inventory document — all resources, their purpose, and their cost implications. (2) Ask Claude to review the configuration against the relevant Well-Architected Framework pillars: security, reliability, performance efficiency, and cost optimisation. (3) Ask Claude to identify any security risks — overly permissive IAM policies, unencrypted storage, missing VPC endpoint configurations. (4) Ask Claude to generate remediation recommendations for the top 3 issues found.

Notes for Nitin Given your background in ISO/IEC 27001 ISMS, the security review component of this exercise is especially valuable. Claude Code can identify common cloud security misconfigurations quickly — IAM policy issues, storage bucket permissions, network security group rules — and generate the corrected configurations. Think of it as a first-pass security audit that surfaces obvious issues before your expert review.

Task 24 · 4 hrs

Exercise: Multi-Environment Configuration Management

Use Claude Code to generate, audit, and maintain your multi-environment configuration strategy across development, staging, and production.

Resources

- Spring Boot Profiles Documentation: https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.profiles

Exercise For one of your projects with multiple deployment environments: (1) Ask Claude to audit your current application.yml and environment-specific override files for configuration inconsistencies between environments. (2) Ask Claude to identify any secrets that should not be in configuration files and should be moved to Kubernetes Secrets or Vault. (3) Ask Claude to generate a configuration validation checklist — things that must be verified before promoting from staging to production. (4) Ask Claude to generate a configuration documentation page describing all configuration properties, their purpose, and their valid values.

Notes for Nitin Configuration management across environments is one of the most common sources of production incidents in microservice systems. Claude Code's ability to read all your configuration files simultaneously and identify inconsistencies is one of the highest-value use cases for your role — it does in seconds what would take an engineer an hour of careful manual comparison.

Milestone 7: Team Leadership and Code Review Workflows

14 hrs · 4 tasks *Experience goal: "I use Claude Code to multiply my effectiveness as a technical leader — conducting better code reviews, writing better specifications, and enabling my team to work to higher standards."*

Task 25 · 3 hrs

Exercise: AI-Augmented Code Review

Use Claude Code as your first-pass code reviewer — running systematic checks before you apply your architectural judgement, so your reviews are faster and more thorough.

Resources No external resources required — apply to real PRs from your team.

Exercise Take 2–3 recent PRs from your team (or simulate with recent commits). For each, ask Claude to: (1) Review for adherence to your architectural conventions (does it follow the package structure, exception handling, and API design standards in your CLAUDE.md?). (2) Review for security issues — improper input validation, missing authentication checks, insecure configuration. (3) Review for performance concerns — N+1 queries, missing caching, synchronous calls where async would be appropriate. (4) Generate a structured code review

comment set you can post directly. Compare Claude's review with what you would have written manually. What did it catch that you would have missed? What did it miss?

Notes for Nitin Build a /review custom command (Task 31) that automates this review workflow. Once it is working reliably, you can run it on every PR before you read it — your review then focuses on architectural and design concerns that Claude cannot evaluate, which is exactly where your 12 years of experience adds the most value.

Task 26 · 3 hrs

Exercise: Junior Developer Mentoring Acceleration

Use Claude Code to generate learning exercises, code examples, and architectural explanations that help you mentor your junior team members more effectively.

Resources No external resources required — use your current team mentoring context.

Exercise Identify 2–3 concepts or patterns that your junior developers currently struggle with — for example, Spring Security configuration, Keycloak integration, or Kubernetes manifest structure. For each: (1) Ask Claude to generate a clear, well-commented code example that demonstrates the pattern correctly. (2) Ask Claude to generate 3 common mistakes junior developers make with this pattern and how to identify and correct them. (3) Ask Claude to generate a self-assessment exercise the junior developer can complete to verify their understanding. Review each output and refine before sharing with your team.

Notes for Nitin This is one of the most underappreciated use cases for senior engineers. The time you spend explaining the same Spring Security or Keycloak pattern to each new team member can be replaced by a Claude-generated, expert-reviewed guide that every developer gets access to from day one. Your role shifts from explaining to reviewing — which is a better use of your experience.

Task 27 · 4 hrs

Exercise: Error Recovery and the 3-Strike Rule for Architecture Work

Build your personal error recovery playbook with examples specific to your Solution Architect work — complex integrations, multi-service designs, and domain-specific implementations.

Resources

- Debugging with Claude Code (Official Docs): https://docs.anthropic.com/en/docs/claude-code/troubleshooting

Exercise Practice 4 recovery patterns with architecture-specific scenarios: (1) Gap Description — "The generated Spring Security configuration handles the token validation correctly but does not implement the resource server role mapping from Keycloak. Expected: roles are extracted from the 'realm_access.roles' claim. Got: standard Spring Security authority mapping. Please fix only the role mapping without changing the token validation." (2) Fresh Session with tighter constraints — /clear and reframe a complex Keycloak or SSI integration with more explicit domain context. (3) 3-Strike Rule — when Claude generates incorrect Argo Workflow YAML 3 times, switch completely: "Ignore the implementation. Describe the workflow logic in plain English first — the steps, their dependencies, and the error handling requirements." (4) Rollback — use Git to revert a multi-service change that was incorrectly generated and restart from the last known good state.

Notes for Nitin For SSI and Catena-X work specifically, the 3-strike rule is your most important recovery tool. These are domains where Claude's training data is limited. If it is generating incorrect DID resolution logic or wrong EDC connector configurations after 3 attempts, do not persist — change your approach to: (1) ask Claude to explain the specification first, (2) ask Claude to describe what the code needs to do in plain language, (3) then ask for the implementation. This three-step approach consistently produces better results than three attempts at direct code generation.

Task 28 · 4 hrs

Exercise: Multi-Session Architecture Projects

Design and execute a multi-session workflow for a complex architecture task — one that spans multiple services, multiple sessions, and requires consistent context across all of them.

Resources

- Nick Saraev's Claude Code Masterclass — Project Management Segment (YouTube): https://www.youtube.com/watch?v=6VakQSJFTZo

Exercise Identify a real architecture task that will require 3+ sessions — for example, implementing a new cross-cutting feature across multiple services, or migrating a component to a new integration pattern. Design your session routine: how you orient Claude at the start of each session (which CLAUDE.md you load, what you summarise from previous sessions), how you update CLAUDE.md at the end of each session (decisions made, files changed, constraints discovered), and how you manage the transition between sessions. Execute the first two sessions and document how well context carried.

Notes for Nitin Multi-session discipline is especially important for your work given the complexity of your projects. A Catena-X integration or SSI workflow implementation that spans three sessions needs a CLAUDE.md update at the end of each session that captures: which

components were built, which interfaces were established, and what the next session needs to know. Without this, each session starts from scratch — with it, each session builds directly on the last.

Milestone 8: Advanced Capabilities and Custom Workflows

16 hrs · 4 tasks *Experience goal: "I have built the custom tools and workflows that make Claude Code a permanent part of my daily practice as a Solution Architect."*

Task 29 · 4 hrs

Exercise: Build Your Custom Slash Command Library

Build a library of custom slash commands that automate your most repetitive architectural tasks into single commands.

Resources

- Custom Slash Commands (Official Docs): https://docs.anthropic.com/en/docs/claude-code/slash-commands
- Hooks Documentation (Official Docs): https://docs.anthropic.com/en/docs/claude-code/hooks

Exercise Create at least 3 custom slash commands for your Solution Architect workflow. Suggestions: (1) /scaffold — given a service name and brief description, generates a complete Spring Boot microservice scaffold following your CLAUDE.md conventions. (2) /review — performs an architectural code review on the current diff, checking conventions, security, and performance concerns. (3) /adr — given a decision description, generates a formatted Architecture Decision Record. Commit all commands to .claude/commands/. Test each on 3 real scenarios. Refine until output is consistently production-quality.

Notes for Nitin The /scaffold command is your highest-leverage tool. Once it reliably generates a new Spring Boot service that follows all your project conventions from a single command, the time from "we need a new service" to "here is the scaffold ready for feature implementation" drops from hours to minutes. This directly accelerates your team's delivery capacity.

Task 30 · 4 hrs

Exercise: MCP Server Integration — Connecting Claude Code to Your Tools

Connect Claude Code to the tools you use daily — Jira, GitHub/GitLab, and potentially AWS — to create end-to-end automated workflows.

Resources

- MCP Integration Guide (Official Docs): https://docs.anthropic.com/en/docs/claude-code/mcp
- MCP Registry: https://registry.modelcontextprotocol.io

Exercise Browse the MCP Registry. Identify 3 MCP servers relevant to your workflow: GitHub or GitLab (PR descriptions, branch management), Jira (create tickets from architectural analysis), and any AWS or infrastructure-related server. For each: (1) Write what workflow you would enable and what the before/after looks like. (2) Set up at least one if your team has approved MCP servers. (3) Test the integration with a real task — for example, asking Claude to create a Jira ticket for a new architectural decision with the full ADR content included.

Notes for Nitin The GitHub/GitLab MCP server is your highest-value integration. Being able to ask Claude to "review this PR, generate a structured review comment set, and post it to GitHub" directly from a Claude Code session removes the context switching between your terminal and your browser that currently fragments your review workflow.

Task 31 · 4 hrs

Exercise: Security and Compliance Review Workflows

Given your involvement in ISO/IEC 27001:2013 ISMS maintenance, use Claude Code to generate security reviews, compliance documentation, and threat modelling outputs.

Resources

- OWASP Top 10: https://owasp.org/www-project-top-ten/
- Spring Security Best Practices: https://docs.spring.io/spring-security/reference/

Exercise For one of your current projects: (1) Ask Claude to perform an OWASP Top 10 review of a service — for each applicable category, identify whether the service has mitigations in place and where gaps exist. (2) Ask Claude to generate a threat model for a specific workflow — identifying assets, threats, mitigations, and residual risk. (3) Ask Claude to generate the security section of a technical specification for an upcoming feature, including authentication requirements, data classification, and access control design. (4) Ask Claude to review your Keycloak realm configuration for common security misconfigurations.

Notes for Nitin Claude Code is a valuable first-pass security reviewer — it consistently identifies common vulnerabilities, missing input validation, and insecure configuration patterns. It is not a replacement for a formal security audit or your ISMS processes, but it raises the baseline security quality of every piece of code your team ships before it ever reaches a formal review. Add a standing instruction to your CLAUDE.md: "Always flag any endpoint that handles PII, any dynamic query construction, and any configuration that sets security headers or CORS policies."

Task 32 · 4 hrs

Exercise: Open Source Contribution Acceleration

You actively contribute to the Eclipse Tractus-X open-source project. Use Claude Code to accelerate your open-source work — understanding unfamiliar codebases, generating contributions, and writing documentation.

Resources

- Eclipse Tractus-X GitHub: https://github.com/eclipse-tractusx
- Contributing to Tractus-X: https://eclipse-tractusx.github.io/docs/oss/how-to-contribute

Exercise Choose a contribution task in the Tractus-X project — a bug fix, a new feature, or documentation improvement. Use Claude Code to: (1) Understand the relevant part of the Tractus-X codebase by asking Claude to explain the component's architecture and design decisions. (2) Plan the contribution using /plan before writing any code. (3) Generate the implementation following the project's existing patterns. (4) Generate the PR description including the problem being solved, the approach taken, and how to test the change. Compare the quality of Claude-assisted contribution versus your previous manual approach.

Notes for Nitin Open-source contribution is one of the use cases where Claude Code's codebase comprehension capability provides the most value. Unfamiliar codebases that would take significant time to understand manually become navigable quickly when you can ask Claude to explain component relationships, trace request flows, and identify the right extension points for your contribution.

Milestone 9: Team Standards and Knowledge Multiplication

14 hrs · 4 tasks *Experience goal: "My Claude Code learnings have become team standards that multiply my effectiveness across every engineer I lead."*

Task 33 · 3 hrs

Study: Team Best Practices, Security, and Anti-Patterns

Study the documented team-level best practices for Claude Code and translate them into standards specific to your team's architecture and security requirements.

Resources

- Team Best Practices (Official Docs): https://docs.anthropic.com/en/docs/claude-code/best-practices
- Security and MCP Servers (Official Docs): https://docs.anthropic.com/en/docs/claude-code/security
- Claude Code Ultimate Guide — Anti-Patterns: https://github.com/FlorianBruniaux/claude-code-ultimate-guide/blob/main/guide/ultimate-guide.md

Exercise Read all three resources. Write in your Notes: (1) 3 team standards you want to establish for Claude Code use across your engineering team — for example, "all Claude-generated code must pass /review before being included in a PR." (2) 1 security concern specific to your team's work (SSI credentials, Keycloak secrets, AWS credentials) that every developer must understand before using Claude Code. (3) 1 anti-pattern you have already noticed in your own sessions that the team should avoid. (4) How you would present these standards to your team in a way that positions Claude Code as an accelerator rather than a replacement for engineering judgement.

Notes for Nitin As Solution Architect and technical team lead, you are the natural owner of your team's Claude Code standards. The security concern around Keycloak secrets, DID private keys, and AWS credentials is especially important — Claude Code reads your local filesystem and these values must never appear in prompts or be committed to repositories. Document this explicitly in your team's Claude Code standards.

Task 34 · 3 hrs

Exercise: Build the Team's CLAUDE.md Library

Create a set of shared, version-controlled CLAUDE.md files that any engineer on your team can use immediately — covering all active projects.

Resources No external resources required — documentation and leadership exercise.

Exercise Review the CLAUDE.md files you built in Milestone 2. Create team versions that: (1) Remove any personal preferences that should remain individual. (2) Add project conventions that other engineers need to know but may not be explicitly documented anywhere else. (3) Add a "common mistakes" section for each project based on issues you have seen in code reviews. (4) Commit to a shared repository with a README explaining what each CLAUDE.md covers and when to use it.

Notes for Nitin The "common mistakes" section is a unique contribution that only you can write — it captures the patterns of errors you have seen across dozens of code reviews. When every engineer on your team starts a session with a CLAUDE.md that includes those patterns as "never do" items, you effectively embed your code review knowledge into every Claude session your team runs.

Task 35 · 4 hrs

Exercise: Build the Team's Architecture Prompt Library

Create a shared prompt library of the most effective prompts for Solution Architecture, Spring Boot development, and cloud-native work — a resource that multiplies your expertise across the whole team.

Resources No external resources required — build and share with your team.

Exercise Create a shared document titled "smartSense Claude Code Architecture Prompt Library." Organise by category and add at least 12 prompts with a "Why this works" note for each. Suggested categories:

- Microservice Scaffolding
- Spring Security and Keycloak Integration
- SSI/DID Component Development
- Kubernetes and Helm Chart Generation
- Argo Workflow Design
- AWS/Azure Infrastructure Review
- Architecture Decision Records
- Cross-Service Refactoring
- API Contract Review
- Security and Compliance Review Share with your team and invite contributions.

Notes for Nitin The SSI/DID and Catena-X categories will be unique to your experience — no other engineer in most organisations will have these prompts. Contributing them to the shared library means every developer who works on Tractus-X or SSI-related features has access to your architectural knowledge in prompt form.

Task 36 · 4 hrs

Exercise: Establish AI-First Architecture Practices for Your Team

Design and document your team's AI-first development practices — the workflow standards that make Claude Code a permanent part of how your team designs, builds, and ships software.

Resources No external resources required — leadership and documentation exercise.

Exercise Create the "smartSense AI-First Architecture Playbook." Include: (1) When to use Claude Code — specific tasks where it adds highest value (scaffolding, cross-cutting changes, architecture documentation, security review, open-source contribution). (2) When not to trust Claude Code without expert review — outputs that always require architectural judgement (security configurations, SSI/DID implementations, cross-service contract changes). (3) The standard development workflow: Claude generates → /review → architect review → commit → PR description generated by Claude. (4) The CLAUDE.md maintenance standard — who owns each project's CLAUDE.md, when to update it, what must be included. (5) The approved MCP server list and security guidelines for MCP usage. Present to your team and iterate based on feedback.

Notes for Nitin This playbook is the highest-leverage deliverable of your entire onboarding. It transforms 160 hours of personal learning into a permanent team practice. Every engineer you lead — now and in the future — benefits from the standards you establish here. As the architect who designed the systems they build, your standards carry the weight of deep contextual knowledge. Document them clearly and maintain them actively.

Milestone 10: Capstone — Build Something Real

14 hrs · 4 tasks *Experience goal: "I have completed a significant architecture or development project using Claude Code end-to-end and can measure and articulate the impact on my work and my team."*

Task 37 · 2 hrs

Capstone Prep: Design the Capstone Project

Your capstone should be a production-bound deliverable that demonstrates the full depth of what you have learned — not just code generation, but architectural thinking, documentation, team standards, and delivery impact.

Resources No external resources required — review your current backlog.

Exercise Identify a capstone project that: (1) Is a real deliverable you are accountable for. (2) Spans at least 3 of your strongest milestones — for example, service scaffolding plus architecture documentation plus team standards, or SSI integration plus security review plus specification writing. (3) Will take 8–12 hours with Claude Code versus significantly more manually. (4) Produces something concrete and demonstrable.

Strong candidates for your role:

- Complete design and implementation of a new microservice for Cofinity-X or Certif-ID — scaffold, integration, Kubernetes deployment, documentation, ADR, and team specification
- A full architectural review of an existing service — security audit, API contract review, documentation generation, and remediation plan
- A new SSI or Catena-X integration — design, implementation, documentation, and contribution to Eclipse Tractus-X

Notes for Nitin Choose something you will be proud to demo. Your capstone demonstrates what AI-first Solution Architecture looks like at a senior level — and sets the standard for how every engineer on your team should aspire to work.

Task 38 · 8 hrs

Capstone: Build the Project

Execute your capstone project applying everything from Milestones 1–9.

Resources No external resources required — this is real project work.

Exercise Execute your capstone applying: (1) Your project CLAUDE.md for every session — loading the correct one for each service or component. (2) /plan before every multi-file or multi-service change. (3) Your /scaffold, /review, and /adr custom commands. (4) The staged approach — one component at a time, reviewed and committed before moving to the next. (5) Session documentation — CLAUDE.md updated at the end of each session. (6) Full Git workflow automation — commit messages and PR description generated by Claude.

Maintain a running log: every prompt that generated high-quality output, every correction made, every time Plan Mode prevented an architectural mistake, and hourly time tracking versus estimated manual effort.

Notes for Nitin At your experience level, the most valuable log entry is not "Claude saved me time on boilerplate" — it is "Claude's architectural suggestion in the plan surfaced a design issue I would have caught in code review two weeks from now." Track these moments. They are the evidence that Claude Code changes not just your execution speed but your design quality.

Task 39 · 2 hrs

Capstone: Measure, Document, and Write Your Before/After Story

Produce the concrete evidence of what Claude Code achieves for a Solution Architect working on complex distributed systems.

Resources No external resources required.

Exercise Write a Before/After impact report covering: (1) What you built — the specific deliverable, its architectural complexity, and the systems it affects. (2) Time comparison — estimated manual time versus actual Claude Code-assisted time for each component (design, scaffold, integration, documentation, review). (3) Quality comparison — what architectural issues did Plan Mode surface before they became code problems? What percentage of generated code was production-ready on first or second attempt? (4) Team impact — the CLAUDE.md library, the prompt library, and the AI-First Architecture Playbook you created. (5) What you would do differently in the next 160 hours.

Notes for Nitin Be specific with numbers. "Scaffolded a complete Spring Boot microservice with Keycloak integration, Kubernetes manifests, and Swagger documentation in 45 minutes versus an estimated 4 hours manually" is compelling evidence. Your data will shape how the rest of your organisation thinks about AI-first architecture.

Task 40 · 2 hrs

Capstone: Present at the Showcase and Lead the Knowledge Transfer

Present your capstone at the team showcase and position yourself as the team's AI-first architecture authority.

Resources No external resources required — prepare your presentation.

Exercise Prepare a 10-minute presentation covering: (1) The architectural deliverable you built — show the actual service, manifests, or documentation. (2) A live demonstration — show your /scaffold command generating a new service, your /review command reviewing a real diff, and your /adr command generating an architecture decision record. (3) Your Before/After numbers. (4) The team assets you created — CLAUDE.md library, prompt library, and playbook. (5) Your honest assessment — where Claude Code adds the most value for architects, and where architectural expertise is irreplaceable. (6) An invitation for every engineer to contribute to the team's shared resources.

End with: "Here is what I am still doing manually that Claude Code cannot help with, and here is the architectural judgement that I will always apply regardless of what the tool generates." This framing establishes your role as architect and director — not just an AI tool user.